NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC1007/CE1007/CZ1007: Data Structures & Algorithms

Linked List: Variations



Presented by Dr Owen Noel Newton Fernando

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Welcome to this lecture on Linked List Variations, where we'll build on our understanding of basic linked lists and explore how their structure can be extended for different requirements. We'll focus on doubly linked lists and circular linked lists, looking at how they are organised, how common operations such as traversal, insertion and deletion work, and where these variations can be useful in practical applications.



Objectives

- Introduce the structure and characteristics of doubly linked lists.
- Explain forward and backward traversal in a doubly linked list.
- Examine search, counting, insertion, and deletion operations.
- Illustrate how node references are updated during list operations.
- Introduce singly and doubly circular linked-list structures.
- Discuss the benefits, limitations, and applications of linked-list variations.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

This lecture will take us beyond the basic singly linked list and explore how linked structures can be adapted when we need more flexibility. We'll begin with doubly linked lists, where connections in both directions change the way we traverse and manage nodes.

We'll work through familiar operations such as searching, counting, insertion and deletion, paying particular attention to how references are updated. Think about what happens when a node has links to neighbours on both sides; changing one node may require several connections to be adjusted correctly.

We'll then explore circular linked lists, including both singly and doubly linked forms, before considering their practical uses. By the end, you should be able to compare these variations and recognise which structure is more suitable for a particular problem.

Intended Learning Outcomes

Upon successful completion of this lecture, students will be able to:

- Differentiate between singly, doubly, and circular linked lists.
- Represent doubly linked-list nodes using object-oriented Python.
- Trace forward and backward traversal through a doubly linked list.
- Implement search, counting, insertion, and deletion operations in Python.
- Analyse how reference updates preserve the integrity of linked-list structures.
- Select an appropriate linked-list variation for a given application.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

By the end of this lecture, you should be able to do more than simply describe the different linked-list variations. You'll be able to distinguish between singly, doubly and circular linked lists, and represent a doubly linked-list structure using object-oriented Python.

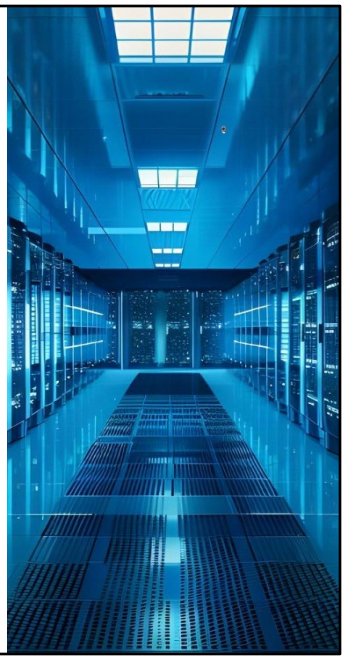
You should also be comfortable tracing movement in both directions and implementing familiar operations such as searching, counting, insertion and deletion. As we work through the code, pay close attention to the references, because maintaining the correct connections between neighbouring nodes is central to preserving the structure of the list.

Beyond implementing these operations, I'd like you to think about when each variation is actually useful. Given a particular application, you should be able to examine its requirements and select a linked-list structure that fits those requirements appropriately.

Contents

- Introduction to Linked-List Variations
- Structure of a Doubly Linked List
- Traversal, Search, and Size Operations
- Insertion in a Doubly Linked List
- Deletion from a Doubly Linked List
- Advantages and Applications of Doubly Linked Lists
- Circular Singly and Doubly Linked Lists
- Advantages and Applications of Circular Linked Lists
- Summary

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted



Here's the path we'll follow through the lecture, starting with a brief introduction to the different linked-list variations and then concentrating on the doubly linked list. We'll first understand its structure and see how having references in both directions changes the way we work with the list.

From there, we'll examine traversal, searching and size operations before spending more time on insertion and deletion. These operations are particularly useful for understanding how references between neighbouring nodes must be updated carefully whenever the structure changes.

In the later part of the lecture, we'll turn to circular linked lists and examine both singly and doubly linked forms. We'll also discuss the advantages, limitations and practical applications of these structures before bringing everything together in a short summary.

Introduction to Linked-List Variations

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

In this section, we'll explore linked-list variations and see why different structures are useful for different situations. We'll begin with doubly linked lists and then move to circular linked lists, focusing on how their connections change the way we traverse, insert, delete and manage nodes compared with the singly linked lists we've already studied.

Why Do We Need Linked-List Variations?

- A singly linked list supports traversal in only one direction.
- Some applications require both forward and backward movement.
- Repeated processing may require traversal to continue from the last node.
- Linked-list variations address these different traversal requirements.
- Each variation changes how nodes are connected through references.
- These structural differences affect operations, efficiency, and applications.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

A singly linked list works well when we only need to move through the nodes in one direction, but that isn't always enough. Think about situations where we may want to move backwards through a sequence or repeatedly cycle through the same collection of items.

Different linked-list structures give us different ways of supporting these behaviours. A doubly linked list adds another reference so that we can move in both directions, while a circular linked list connects the end of the list back into the structure rather than simply stopping there.

Of course, these extra connections come with trade-offs in memory usage and the way operations are implemented. As we explore each variation, we'll look closely at how the references change and when the additional flexibility is actually useful.

Main Linked-List Variations

- A singly linked list connects each node to the next node.
- A doubly linked list connects each node to both neighbouring nodes.
- A circular singly linked list connects the last node to the first node.
- A circular doubly linked list supports circular traversal in both directions.
- Each variation offers different advantages, limitations, and memory requirements.
- The appropriate variation depends on the operations required by the application.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

There are four main linked-list variations that we'll work with, and the difference between them comes down to how their nodes are connected. A singly linked list provides one-way links, while a doubly linked list adds references that allow us to move in either direction.

Circular lists change the structure in a different way by connecting the end of the list back to the beginning. This can be combined with either singly or doubly linked nodes, giving us circular traversal in one direction or in both directions.

These variations aren't simply alternative versions of the same structure; each involves different memory costs and operational behaviour. Choosing between them depends on what the application needs, particularly how we want to traverse, insert, and remove nodes.

Structure and Basic Operations of a Doubly Linked List

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We'll begin by looking closely at the structure of a doubly linked list and how it differs from the singly linked list you already know. From there, we'll work through its basic operations and see how having references in both directions changes traversal, searching, insertion, and deletion.

Doubly Linked List

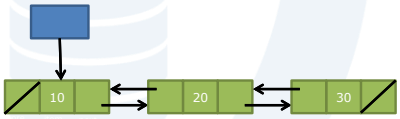
Singly Linked list: Only one link. Traversal of the list is one way only.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```



Doubly Linked List: Each node contains two links, allowing traversal in both forward and backward directions.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.pre = None
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

A doubly linked list builds on the singly linked structure by giving each node an additional reference. Instead of knowing only which node comes next, a node can also keep track of the node that comes before it.

You can see the difference clearly in these two diagrams: the singly linked list allows us to follow the sequence in one direction, whereas the doubly linked list supports movement both forwards and backwards. In our Python representation, this requires an additional pre reference alongside the existing next reference.

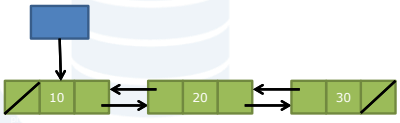
That extra reference gives us more flexibility when navigating and modifying the list, although it also means maintaining more links correctly. As we work through the operations, pay particular attention to how both next and pre references are updated to preserve the structure.

Doubly Linked List: Print

Printing follows the same forward-traversal process used in a singly linked list.

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5         self.pre = None
```

```
1 def print_list(head):
2     current = head
3     while current:
4         print(current.data, end=" -> ")
5         current = current.next
6     print("None")
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Printing a doubly linked list may look as though it should be more complicated because every node has two references, but forward traversal is actually very familiar. If we're travelling from the head towards the end, we only need to follow the next reference.

The function begins with current pointing to the head, and the loop continues while a node is available. At each step, we print the data and then move current to current.next, progressing through the nodes one by one.

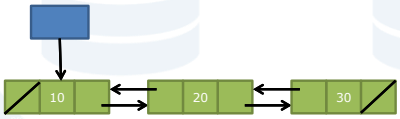
Notice that the pre reference isn't used at all during this operation. It becomes useful when we want to travel backwards, which is one of the main capabilities that distinguishes a doubly linked list from a singly linked list.

Doubly Linked List: Search

Searching follows the same forward-traversal process used in a singly linked list.

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5         self.pre = None
```

```
1 def search(self, data):
2         current = self.head
3         while current:
4             if current.data == data:
5                 return True
6             current = current.next
7         return False
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Searching a doubly linked list can use exactly the same forward-traversal approach we used for a singly linked list. We start at the head and examine each node in sequence, following the next reference as we go.

For each node, the function compares the stored value with the value we're looking for. If they match, we can stop immediately and return True; otherwise, the search continues until there are no more nodes to examine.

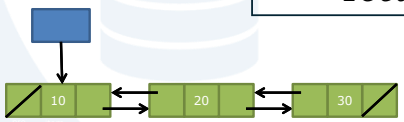
Notice that the pre reference isn't needed for this particular search because we're travelling forwards from the head. The process simply continues from one node to the next until the required value is found or we reach the end of the list.

Doubly Linked List: Size (Count)

- The number of nodes can be determined by traversing the list and maintaining a counter.

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5         self.pre = None
```

```
1 def size(head):
2     count = 0
3     current = head
4     while current:
5         count += 1
6         current = current.next
7     return count
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Counting the nodes in a doubly linked list is quite straightforward, and the process is very similar to what we used with a singly linked list. We begin at the head and move through the list one node at a time using the next reference.

The counter starts at zero, and each time we visit a node, we increase it by one. We then move current to the next node and repeat the process until current becomes None, which tells us that we've reached the end.

Notice that the pre reference isn't required here because we only need to travel forwards through the list. Once the traversal finishes, the value stored in count gives us the total number of nodes currently in the list.

Insertion in a Doubly Linked List

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

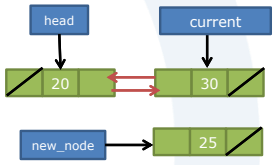
Insertion in a doubly linked list is a little more involved than in a singly linked list because each node has references in both directions. When we insert a new node, we need to update these references carefully so that the forward and backward connections remain consistent throughout the list.

Doubly Linked List: Insertion (middle)

Here, we insert a new node containing 25 between nodes 20 and 30 by carefully updating the surrounding references.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.pre = None
```

```
1   new_node = Node(data)
2   current = self.head
3   for i  in range(index):
4       current = current.next
5
6   new_node.pre = current.pre
7   new_node.next = current
8   current.pre = new_node
9   new_node.pre.next = new_node
10  self.size += 1
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

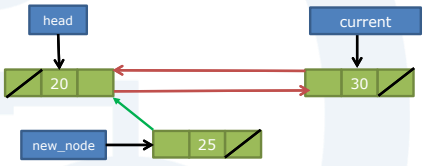
This example shows what happens when we insert a new node into the middle of a doubly linked list. Here, we want to place the new node containing 25 between the existing nodes 20 and 30, so both directions of the list need to be preserved.

We first move current to the node at the required position, which in this example is node 30. The new node's pre reference is connected to 20, while its next reference is connected to current, giving the new node links to both neighbours.

We then update the surrounding nodes so that 30 points back to the new node and 20 points forward to it. Once these four references are correctly established, the new node becomes part of the list and we increase the stored size by one.

Doubly Linked List: Insertion (middle)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.pre = None
```




```
1   new_node = Node(data)
2   current = self.head
3   for i  in range(index):
4       current = current.next
5
6   new_node.pre = current.pre
7   new_node.next = current
8   current.pre = new_node
9   new_node.pre.next = new_node
10  self.size += 1
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Here we focus on the first reference update needed to insert node 25 between nodes 20 and 30. Before changing any of the existing links, we first connect the new node backwards to the node that comes before current.

Since current points to node 30, current.pre points to node 20. The statement new_node.pre = current.pre copies that reference, so new_node.pre now points to node 20, exactly as shown by the green arrow.

Notice that the existing connections between nodes 20 and 30 have not been changed yet. We're adding the required references one at a time, and the next step will connect the new node forward to current.

Doubly Linked List: Insertion (middle)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.pre = None
```




```
1   new_node = Node(data)
2   current = self.head
3   for i  in range(index):
4       current = current.next
5
6   new_node.pre = current.pre
7   new_node.next = current
8   current.pre = new_node
9   new_node.pre.next = new_node
10  self.size += 1
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

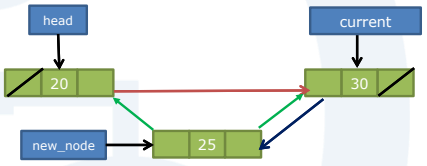
The second reference update connects the new node forwards to current, which is node 30. We already connected 25 backwards to 20, so we now need to establish its connection in the other direction.

The statement new_node.next = current makes the next reference of node 25 point directly to node 30. This is represented by the second green arrow, giving the new node references to both of its intended neighbours.

Notice that the original links between nodes 20 and 30 are still unchanged at this stage. Node 25 knows about both neighbours, but those neighbouring nodes have not yet been fully redirected to include 25 in the list.

Doubly Linked List: Insertion (middle)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.pre = None
```




```
1   new_node = Node(data)
2   current = self.head
3   for i  in range(index):
4       current = current.next
5
6   new_node.pre = current.pre
7   new_node.next = current
8   current.pre = new_node
9   new_node.pre.next = new_node
10  self.size += 1
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

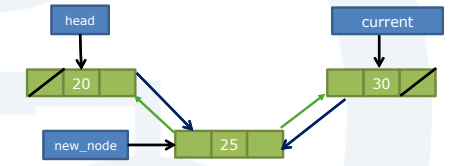
At this stage, node 25 already has references to both of its intended neighbours, nodes 20 and 30. We now need to update node 30 so that its backward reference recognises 25 as its new previous node.

The statement current.pre = new_node changes the pre reference of current, which is node 30. Instead of pointing backwards to node 20, it now points to node 25, as shown by the blue arrow.

Node 25 and node 30 are therefore connected correctly in both directions. However, node 20 still points forward to 30, so one final reference update is needed to complete the insertion.

Doubly Linked List: Insertion (middle)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.pre = None
```




```
1   new_node = Node(data)
2   current = self.head
3   for i  in range(index):
4       current = current.next
5
6   new_node.pre = current.pre
7   new_node.next = current
8   current.pre = new_node
9   new_node.pre.next = new_node
10  self.size += 1
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

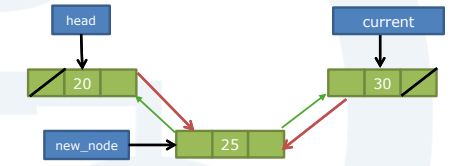
The last reference update connects node 20 forward to the new node 25. At this stage, node 25 already points backwards to 20 and forwards to 30, while node 30 points backwards to 25.

The statement new_node.pre.next = new_node first uses new_node.pre to reach node 20. It then changes the next reference of node 20 so that it points to new_node, which is node 25, as shown by the blue arrow.

With this update, the new node is fully integrated between its two neighbours. We can now traverse 20 → 25 → 30 forwards and 30 → 25 → 20 backwards, completing the reference updates required for the insertion.

Doubly Linked List: Insertion

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.pre = None
```




```
1   new_node = Node(data)
2   current = self.head
3   for i  in range(index):
4       current = current.next
5
6   new_node.pre = current.pre
7   new_node.next = current
8   current.pre = new_node
9   new_node.pre.next = new_node
10  self.size += 1
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

At this stage, all the required references have been updated, so node 25 is correctly connected between nodes 20 and 30. The structural part of the insertion is complete, with the list supporting traversal in both directions.

We still need to update the information maintained about the list itself. Since one new node has been inserted, the statement self.size += 1 increases the stored size by one so that it matches the actual number of nodes.

Keeping this value accurate is necessary because other operations may rely on self.size when working with the list. With the references updated and the size increased, the insertion operation is complete.

Doubly Linked List: Insertion (middle)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.pre = None
```




```
1   new_node = Node(data)
2   current = self.head
3   for i  in range(index):
4       current = current.next
5
6   new_node.pre = current.pre
7   new_node.next = current
8   current.pre = new_node
9   new_node.pre.next = new_node
10  self.size += 1
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

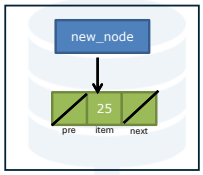
Once the insertion method finishes, its local variables such as current and new_node go out of scope, so those variable bindings are no longer available. However, the newly inserted node remains in the list because the surrounding nodes now maintain references to it.

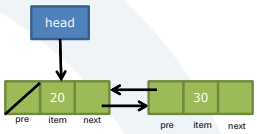
Looking at the completed structure, we can see that node 25 has been successfully inserted between nodes 20 and 30. Node 20 links forward to 25, while node 30 links backwards to 25, giving us the correct connections in both directions.

Node 25 also holds references to both neighbouring nodes, so the doubly linked structure remains intact. With the stored size increased by one, the middle insertion operation is now fully complete.

Doubly Linked List: Insertion

```
1   def insert_at(self, index, data):
2       # If index is invalid
3       if index < 0 or index > self.size:
4           raise ValueError("Invalid position")
5
6       # Create new node
7       new_node = Node(data)
8
9       # If inserting at beginning
10      if index == 0:
11          new_node.next = self.head
12          if self.head:
13              self.head.pre = new_node
14          self.head = new_node
15
16      # Inserting at middle or end
17      else:
18          current = self.head
19          # Traverse to position
20          for i in range(index-1):
21              current = current.next
22
23          # Link new node
24          new_node.pre = current
25          new_node.next = current.next
26          if current.next:
27              current.next.pre = new_node
28          current.next = new_node
          self.size += 1
```





© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

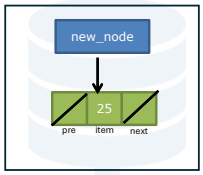
This first step prepares the new node that we want to insert into the doubly linked list. In our example, the existing list contains nodes 20 and 30, and we want to introduce a new node containing 25.

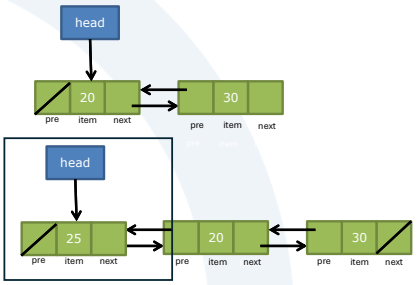
The statement new_node = Node(data) creates that node using the value supplied to the insertion method. At this moment, its pre and next references are both None, so node 25 is still completely separate from the existing list.

You can see this clearly in the diagram, where new_node points to 25 while the original list remains unchanged. We'll connect this new node to the appropriate neighbouring nodes in the following steps.

Doubly Linked List: Insertion

```
1   def insert_at(self, index, data):
2       # If index is invalid
3       if index < 0 or index > self.size:
4           raise ValueError("Invalid position")
5
6       # Create new node
7       new_node = Node(data)
8
9       # If inserting at beginning
10      if index == 0:
11          new_node.next = self.head
12          if self.head:
13              self.head.pre = new_node
14          self.head = new_node
15
16      # Inserting at middle or end
17      else:
18          current = self.head
19          # Traverse to position
20          for i in range(index-1):
21              current = current.next
22
23          # Link new node
24          new_node.pre = current
25          new_node.next = current.next
26          if current.next:
27              current.next.pre = new_node
28          current.next = new_node
        self.size += 1
```





© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

22

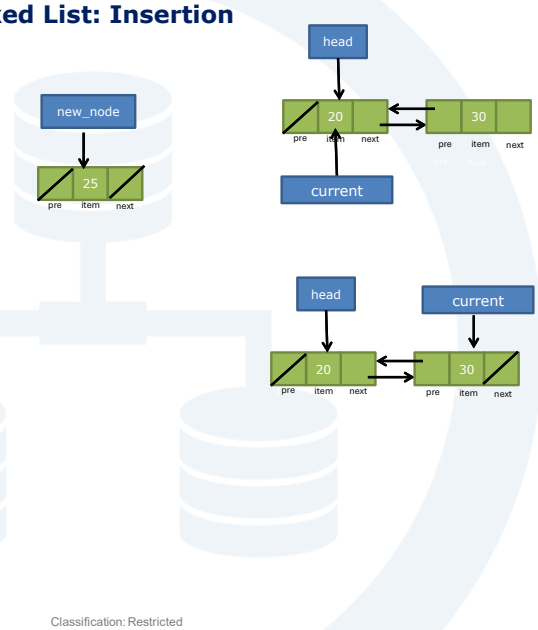
Here we're handling the special case where the new node is inserted at index zero, which means it must become the first node in the doubly linked list. In our example, node 25 is being placed before the existing first node, which contains 20.

The code first connects new_node.next to the node currently referenced by head, preserving the forward path to the existing list. If the list is not empty, the pre reference of the existing first node is then changed to point backwards to the new node, giving us the required connection in both directions.

Finally, self.head = new_node updates the head reference so that it points to node 25, making 25 the new first node. The resulting sequence is therefore 25, 20, 30, exactly as shown in the diagram.

Doubly Linked List: Insertion

```
1   def insert_at(self, index, data):
2       # If index is invalid
3       if index < 0 or index > self.size:
4           raise ValueError("Invalid position")
5
6       # Create new node
7       new_node = Node(data)
8
9       # If inserting at beginning
10      if index == 0:
11          new_node.next = self.head
12          if self.head:
13              self.head.pre = new_node
14          self.head = new_node
15
16      # Inserting at middle or end
17      else:
18          current = self.head
19          # Traverse to position
20          for i in range(index-1):
21              current = current.next
22
23          # Link new node
24          new_node.pre = current
25          new_node.next = current.next
26          if current.next:
27              current.next.pre = new_node
28          current.next = new_node
          self.size += 1
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

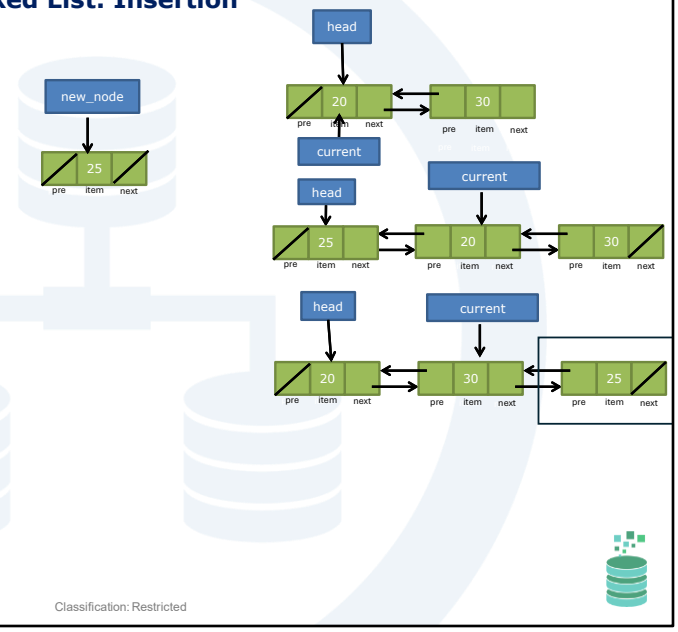
Here we're dealing with insertion somewhere after index zero, so the first task is to reach the correct position in the existing list. We create the local variable current and initially make it refer to the first node, which is the node currently referenced by head.

The for loop then moves current forwards through the list by repeatedly following the next reference. Since we use range(index - 1), the traversal stops at the node immediately before the position where the new node will be inserted.

In the example shown, current starts at node 20 and moves forward to node 30, which is the node immediately before the required insertion position. At this stage, we're only locating that position; none of the links in the list have been changed yet.

Doubly Linked List: Insertion

```
1   def insert_at(self, index, data):
2       # If index is invalid
3       if index < 0 or index > self.size:
4           raise ValueError("Invalid position")
5
6       # Create new node
7       new_node = Node(data)
8
9       # If inserting at beginning
10      if index == 0:
11          new_node.next = self.head
12          if self.head:
13              self.head.pre = new_node
14          self.head = new_node
15
16      # Inserting at middle or end
17      else:
18          current = self.head
19          # Traverse to position
20          for i in range(index-1):
21              current = current.next
22
23          # Link new node
24          new_node.pre = current
25          new_node.next = current.next
26          if current.next:
27              current.next.pre = new_node
28          current.next = new_node
            self.size += 1
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

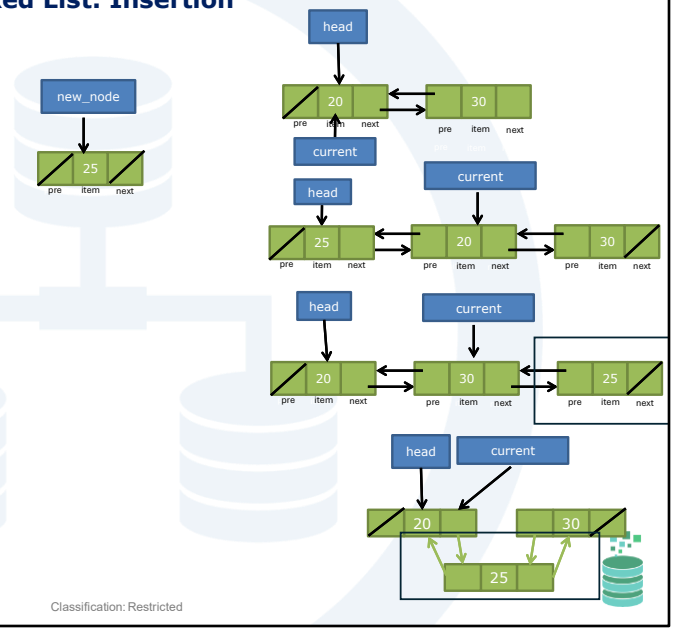
Here we're using the same code section for insertion at the middle or the end, but this example focuses specifically on adding a node at the end. After traversal, current refers to the existing last node, which contains 30.

We first set new_node.pre = current, so node 25 points backwards to node 30. Since current.next is None, new_node.next also becomes None, and the if current.next condition is false, so there is no following node whose pre reference needs updating.

Finally, current.next = new_node connects node 30 forwards to node 25, making 25 the new last node. The size is then increased by one, completing the insertion at the end of the list.

Doubly Linked List: Insertion

```
1   def insert_at(self, index, data):
2       # If index is invalid
3       if index < 0 or index > self.size:
4           raise ValueError("Invalid position")
5
6       # Create new node
7       new_node = Node(data)
8
9       # If inserting at beginning
10      if index == 0:
11          new_node.next = self.head
12          if self.head:
13              self.head.pre = new_node
14          self.head = new_node
15
16      # Inserting at middle or end
17      else:
18          current = self.head
19          # Traverse to position
20          for i in range(index-1):
21              current = current.next
22
23          # Link new node
24          new_node.pre = current
25          new_node.next = current.next
26          if current.next:
27              current.next.pre = new_node
28          current.next = new_node
            self.size += 1
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Here we're using the same code again, but this time the new node is being inserted in the middle of the list. After the traversal, current refers to node 20, which is the node immediately before the required insertion position.

First, new_node.pre = current connects node 25 backwards to 20, while new_node.next = current.next connects it forwards to 30. Because current.next refers to node 30, the if condition is true, so current.next.pre = new_node changes node 30's backward reference to point to 25.

Finally, current.next = new_node changes node 20's forward reference to point to 25. After increasing self.size, all the required links are in place, giving us the sequence 20, 25, 30.

Deletion from a Doubly Linked List

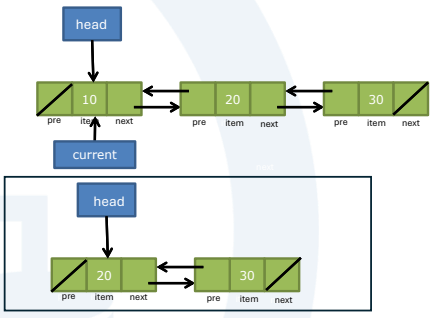
© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Deletion from a doubly linked list requires a little more care than deletion from a singly linked list because each node participates in connections in both directions. When we remove a node, we therefore need to reconnect the surrounding nodes correctly so that both forward and backward traversal continue to work.

We'll look at deletion step by step, considering what happens when the node appears at the beginning, in the middle, or at the end of the list. As with insertion, the order in which we update these references matters because we don't want to lose access to the remaining nodes while changing the structure.

Doubly Linked List: Deletion (Index)

```
1 def remove_at(self, index):
2       # Case 1: Check if the list is empty
3       if self.head is None:
4           print("List is empty")
5           return False
6       # Case 2: Validate index
7       if index < 0 or index >= self.size:
8           print("Invalid index")
9           return False
10      # Case 3: Remove the first node (index 0)
11      if index == 0:
12          self.head = self.head.next
13          if self.head:  # If the list is not empty after removal
14              self.head.pre = None
15          self.size -= 1
16          return True
17      # Case 4: Remove from the middle or end
18      current = self.head
19       for i in range(index): # Traverse to the node at the given index
20          current = current.next
21      # Update pointers to remove the node
22      current.pre.next = current.next
23      if current.next:  # If it's not the last node
24          current.next.pre = current.pre
25      self.size -= 1
26      return True
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

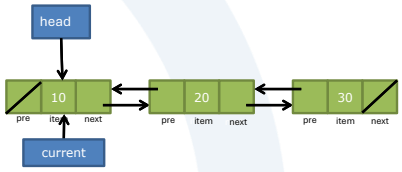
Here we're focusing on deletion at index zero, which means removing the first node from the doubly linked list. In this example, node 10 is the first node, and the head reference currently points to it.

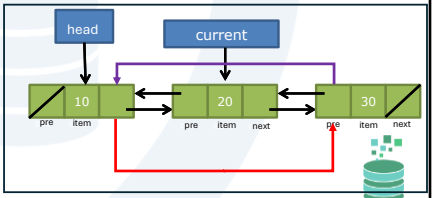
The statement self.head = self.head.next moves the head reference forward so that it now points to node 20. If the list still contains a node after this change, self.head.pre = None removes its backward connection to the node that has just been removed.

Finally, self.size -= 1 reduces the stored number of nodes, and the method returns True to indicate successful deletion. Node 20 is now the first node, with its pre reference set to None, giving the remaining sequence 20, 30.

Doubly Linked List: Deletion (Index)

```
1 def remove_at(self, index):
2       # Case 1: Check if the list is empty
3       if self.head is None:
4           print("List is empty")
5           return False
6       # Case 2: Validate index
7       if index < 0 or index >= self.size:
8           print("Invalid index")
9           return False
10      # Case 3: Remove the first node (index 0)
11      if index == 0:
12          self.head = self.head.next
13          if self.head:  # If the list is not empty after removal
14              self.head.pre = None
15          self.size -= 1
16          return True
17      # Case 4: Remove from the middle or end
18      current = self.head
19       for i in range(index): # Traverse to the node at the given index
20          current = current.next
21      # Update pointers to remove the node
22      current.pre.next = current.next
23      if current.next:  # If it's not the last node
24          current.next.pre = current.pre
25      self.size -= 1
26      return True
```





© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

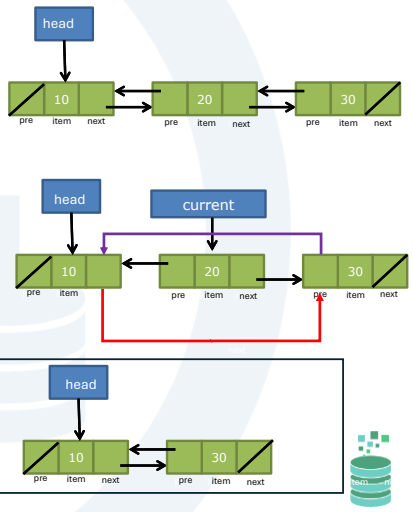
For deletion from the middle of the list, we first need to reach the node that should be removed. Here, current begins from the node referenced by head and moves forward until it reaches node 20, which is the node at the required index.

Once current identifies node 20, we need to reconnect the nodes on either side of it. The statement current.pre.next = current.next changes node 10's next reference to point directly to node 30, bypassing node 20 in the forward direction.

Because node 20 is not the last node, we also set current.next.pre = current.pre, making node 30 point back to node 10. After reducing self.size, node 20 is no longer part of the list, leaving the sequence 10 and 30 correctly connected in both directions.

Doubly Linked List: Deletion (Index)

```
def remove_at(self, index):
    # Case 1: Check if the list is empty
    if self.head is None:
        print("List is empty")
        return False
    # Case 2: Validate index
    if index < 0 or index >= self.size:
        print("Invalid index")
        return False
    # Case 3: Remove the first node (index 0)
    if index == 0:
        self.head = self.head.next
        if self.head:  # If the list is not empty after removal
            self.head.pre = None
        self.size -= 1
        return True
    # Case 4: Remove from the middle or end
    current = self.head
     for i in range(index): # Traverse to the node at the given index
        current = current.next
    # Update pointers to remove the node
    current.pre.next = current.next
    if current.next:  # If it's not the last node
        current.next.pre = current.pre
    self.size -= 1
    return True
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

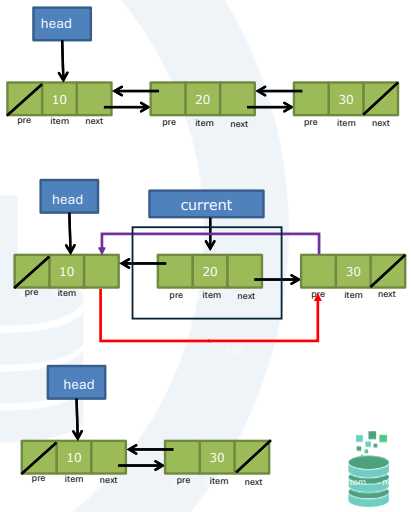
At this stage, the structural changes for the deletion are complete, so we need to make sure the stored size of the list also reflects that change. Whether we delete the first node at index zero, a node from the middle, or the last node, exactly one node has been removed, so self.size must be reduced by one.

Think about why this small update matters: the size variable should always represent the actual number of nodes currently belonging to the linked list. In this example, we started with three nodes, and after removing node 20, only nodes 10 and 30 remain, so the size changes from three to two.

Once the size has been updated, the method can return True to indicate that the deletion was completed successfully. The list is therefore left in a consistent state, with both its references and its recorded size correctly updated.

Doubly Linked List: Deletion (Index)

```
def remove_at(self, index):
    # Case 1: Check if the list is empty
    if self.head is None:
        print("List is empty")
        return False
    # Case 2: Validate index
    if index < 0 or index >= self.size:
        print("Invalid index")
        return False
    # Case 3: Remove the first node (index 0)
    if index == 0:
        self.head = self.head.next
        if self.head:  # If the list is not empty after removal
            self.head.pre = None
        self.size -= 1
        return True
    # Case 4: Remove from the middle or end
    current = self.head
     for i in range(index): # Traverse to the node at the given index
        current = current.next
    # Update pointers to remove the node
    current.pre.next = current.next
    if current.next:  # If it's not the last node
        current.next.pre = current.pre
    self.size -= 1
    return True
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

At this stage, node 20 has been removed from the linked-list structure because nodes 10 and 30 now reference each other directly. However, current is a local variable that still refers to node 20, so the node remains accessible while the method is executing.

Node 20 may still contain its own pre and next references to nodes 10 and 30, but these are outgoing references from node 20. They do not reconnect node 20 to the list, because nodes 10 and 30 no longer hold references back to it.

When the method finishes, the local variable current goes out of scope and its reference to node 20 disappears. Assuming there are no other references to node 20 elsewhere in the program, the node becomes unreachable and Python's automatic garbage collection mechanism can reclaim the memory occupied by that node.

Advantages, Disadvantages, and Applications of Doubly Linked Lists

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We've now seen how a doubly linked list is structured and how insertion and deletion work by updating references between neighbouring nodes. With these operations understood, we can look at the advantages and disadvantages of maintaining links in both directions, before considering some practical applications where doubly linked lists are particularly useful.

Advantages and Disadvantages: Doubly Linked Lists

- **Advantages:**
  - Can be traversed in **either direction** (may be essential for some programs)
  - Some operations, such as deletion and inserting before a node, become easier
- **Disadvantages:**
  - Requires **more space**
  - Requires more reference updates during insertion and deletion
  - Greater chance of having bugs (because more links must be manipulated)

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

A doubly linked list gives us something a singly linked list cannot provide directly: we can move through the structure in either direction. Because each node keeps both next and pre references, operations such as moving backwards, deleting a known node, or inserting before a known node can also be easier.

That flexibility comes with a cost, because every node needs an additional reference, so the list requires more memory. We also have more links to maintain during insertion and deletion, which can mean extra work compared with a singly linked list.

There is also more opportunity for programming errors when several references must be changed in the correct order. So, whether a doubly linked list is suitable depends on whether bidirectional traversal and easier link manipulation justify the additional space and complexity.

Practical Applications - Doubly Linked

- Doubly linked lists support bidirectional traversal and flexible insertion and deletion by maintaining links to both neighbouring nodes.
  - **Browser History**: Each visited page can be represented as a node, allowing users to move backwards to earlier pages and forwards to more recently visited pages.
  - **Undo and Redo**: Application states or user actions can be stored sequentially, enabling movement backwards to undo changes and forwards to restore them.
  - **LRU Cache**: When combined with a hash map, a doubly linked list supports efficient access, insertion, repositioning, and removal of cache entries.
- These applications benefit from efficient movement and updates in both directions without shifting the remaining elements.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Think about situations where moving in both directions through a sequence is useful, because this is where doubly linked lists fit naturally. Browser history is a familiar example: each visited page can be connected to the previous and next pages, supporting movement backwards and forwards.

A similar idea appears in undo and redo operations, where we may move backwards through earlier actions and then forwards again when an action is restored. The two references maintained by each node make this kind of bidirectional movement straightforward.

Doubly linked lists are also commonly used with hash maps when implementing an LRU cache. The hash map provides fast lookup, while the linked list allows entries to be repositioned or removed efficiently without shifting other elements in the structure.

Circular Linked List

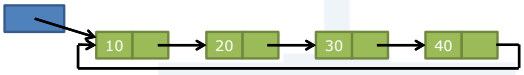
© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We've explored singly and doubly linked lists, so the next variation is the circular linked list. Unlike the structures we've seen so far, a circular linked list connects the final node back into the list, creating a continuous cycle rather than ending with a None reference.

Circular Linked List

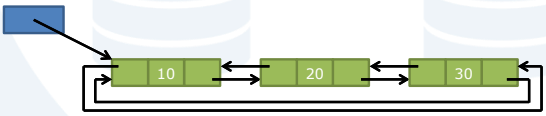
Circular singly linked list

- The next reference of the last node points to the first node.
- Starting from any node, repeated forward traversal eventually returns to that node.



Circular doubly linked list

- The next reference of the last node points to the first node.
- The pre reference of the first node points to the last node.
- This structure supports continuous traversal in both directions.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

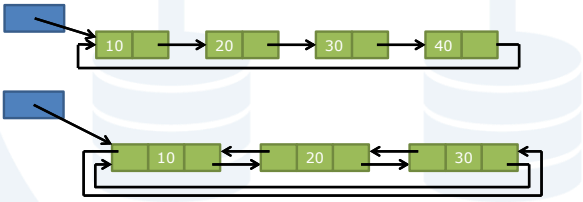
A circular linked list changes one basic feature of the structures we've seen earlier: instead of the final node pointing to None, it connects back to the first node. In a circular singly linked list, this creates a continuous forward path, so repeated traversal eventually brings us back to where we started.

The same idea can be extended to a doubly linked list, but here we maintain connections in both directions. The last node's next reference points to the first node, while the first node's pre reference points back to the last node.

As a result, there is no conventional end marked by a None reference in either traversal direction. We can continue moving around the circular doubly linked list forwards or backwards, provided our traversal logic knows when to stop.

Circular Linked List

- Operations
  - **Display, Search, and Size**: Traversal stops when the current node returns to head, rather than when it reaches None.
  - **Insertion and Deletion**: When the first or last node is affected, head and the circular references connecting the first and last nodes must be updated.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The way we perform operations on a circular linked list is slightly different because there is no None reference marking the end of the structure. For display, search, or size operations, we can begin from the first node and continue traversal until the current reference comes back to the node referenced by head.

Insertion and deletion require a little more care, particularly when the first or last node is involved. Changing either end means we must preserve the circular connection, rather than leaving the structure with a broken link.

For a circular doubly linked list, there are connections in both directions, so both must remain consistent after an update. In practice, the operation is complete only when the neighbouring links, the circular links, and head where necessary have all been updated correctly.

Advantages, Disadvantages, and Applications of Circular Linked Lists

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We've seen how circular linked lists maintain a continuous connection between the first and last nodes, but this structure naturally comes with both benefits and trade-offs. We'll now look at the advantages and disadvantages of maintaining these circular references, before considering some practical situations where circular linked lists are particularly useful.

Advantages and Disadvantages: Circular Linked Lists

- **Advantages:**
  - **Continuous Cyclic Traversal**: Traversal from any node will eventually return to the starting node, making circular lists ideal for applications needing repeated cycling.
- **Disadvantages:**
  - **Risk of Infinite Loops**: Errors in traversal logic can easily lead to infinite loops due to the circular structure, especially if stop conditions are missed.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

A circular linked list is particularly useful when we want traversal to continue repeatedly rather than stop at the end of the list. Because the last node links back to the first, we can keep cycling through the nodes without restarting the traversal manually.

That same circular structure introduces a programming risk, because there is no None reference to tell us naturally that we have reached the end. If the stopping condition is written incorrectly, traversal can continue around the list indefinitely and create an infinite loop.

So when working with circular linked lists, we need to define the termination condition carefully. For example, during a complete traversal, we normally stop when the current reference returns to the node where the traversal began.

Practical Applications - Circular Linked Lists

- **Round-Robin Scheduling**:
  - Operating systems use circular linked lists for round-robin scheduling to cycle through processes, moving to the next in a loop, which ensures each task gets its turn.
- **Circular Buffers**:
  - Circular linked lists are used in circular buffers, which are common in streaming, data buffering, and telecommunications, allowing efficient handling of incoming and outgoing data.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

One natural fit for a circular linked list is any situation where processing needs to repeat continuously through a collection. Instead of reaching the final node and stopping, the structure naturally takes us back to the beginning so that the next cycle can start.

Think about round-robin scheduling, where processes are given turns in sequence. After the last process receives its turn, scheduling continues again from the first, so the circular structure provides a natural way of representing this repeated rotation.

Circular buffers follow a similar idea, although their implementation does not necessarily require a linked list. They repeatedly reuse a fixed logical sequence of positions, making them useful for handling continuous streams of data in areas such as buffering and telecommunications.

Conclusion

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

To conclude, we have explored how linked lists can be extended into doubly and circular structures, and how these variations support different traversal, insertion, deletion, and application requirements.

Conclusion

- In this lecture, we examined the structures, operations, advantages, and applications of doubly and circular linked lists.
- Doubly linked lists store pre (previous) and next references in each node.
- They support both forward and backward traversal through the list.
- Searching, displaying, and counting are similar to singly linked-list operations.
- Insertion and deletion require careful updating of neighbouring node references.
- Circular linked lists connect the last node back to the first node.
- The appropriate linked-list variation depends on the application's traversal and update requirements.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

As we wrap up, it's useful to bring the different linked-list variations together and think about what each structure gives us. Doubly linked lists add pre and next references, allowing us to move through the list in either direction rather than only forwards.

Many familiar operations, such as displaying, searching, and counting nodes, still follow the same general traversal ideas we used with singly linked lists. Insertion and deletion need more attention because references on both sides of the affected position may need to be updated correctly.

Circular linked lists make a different structural change by connecting the last node back to the first. So when choosing a linked-list variation, we should consider how the application needs to traverse and update its data.

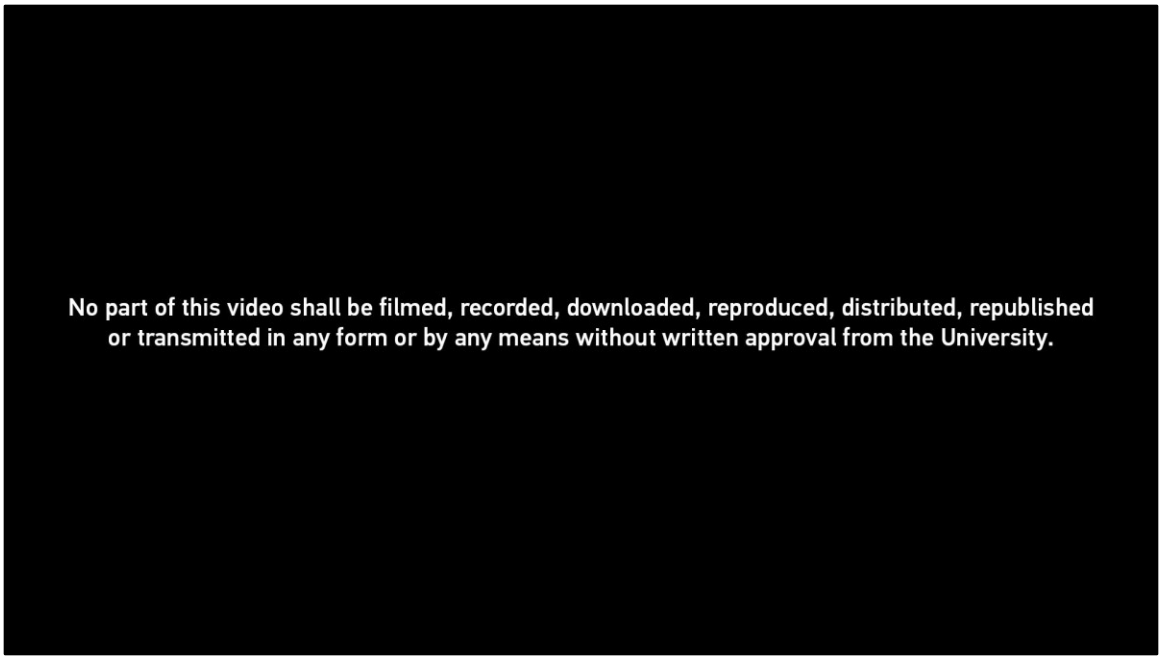
Acknowledgements

Image credits (arranged in order of appearance):

- https://www.freepik.com/premium-ai-image/binary-code-background-made-zeros-ones-representing-essence-digital-technology_383619980.htm
- https://www.freepik.com/premium-photo/worldwide-server-data_260897651.htm
- https://www.freepik.com/premium-photo/worldwide-server-data_260897651.htm
- https://www.istockphoto.com/vector/binary-tree-gm479524481-17312880?clarity=false



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted


No part of this video shall be filmed, recorded, downloaded, reproduced, distributed, republished
or transmitted in any form or by any means without written approval from the University.